

# Development and Validation of a Dexterous Hand System

Final Report

Jincheng Tian

GuangTian Liu

ZhongHao Zhao

May 2026

## Abstract

This report presents a DexHand-based dexterous hand system that integrates mechanical assembly, embedded control, PC-side software, calibration, demonstration, and simulation preparation. Based on the open-source DexHand platform, the project reproduced the main hand–wrist–forearm structure and adapted the control system to an ESP32-S3 + PCA9685 architecture. Custom firmware was developed for command parsing, servo mapping, range limiting, and PWM output, while a Tkinter-based PC-side monitor was implemented for pose control, servo testing, action recording, range calibration, gesture playback, and logging. After all 15 servo channels were calibrated, preset gestures were replayed on the physical prototype, including sequential playback and smooth transitions between saved actions. ROS2 visualization and an initial MuJoCo setup were also explored for future intelligent-control work. This report emphasizes both the system integration process and the boundary between the open-source DexHand resources and our independent engineering contributions.

**Keywords:** DexHand; dexterous hand; ESP32-S3; servo control; Tkinter; ROS 2; MuJoCo.

---

## 1 Introduction

### 1.1 Background and Motivation

Dexterous robotic hands are important end-effectors for humanoid robots and manipulation research. Compared with simple grippers, a multi-finger hand can perform richer gestures and more flexible motions, but it also requires the integration of mechanical structure, actuators, embedded control, software tools, calibration, and simulation.

In this project, DexHand was selected as the open-source reference platform. Instead of redesigning a dexterous hand from the beginning, our goal was to reproduce, adapt, and validate a DexHand-based prototype within our hardware and course constraints.

### 1.2 Project Objective and Scope

The main objective of this project was to build an integrated dexterous hand system from mechanical assembly to motion demonstration. Based on the DexHand reference design, we completed the assembly of the fingers, palm, wrist, forearm, tendons, and servos. We also adapted the control system to an **ESP32-S3 + PCA9685** architecture and developed firmware for command parsing, servo mapping, range limiting, and PWM output.

To support debugging and demonstration, a **Tkinter-based PC-side monitor** was developed. It provides

pose control, single-servo testing, action recording, range calibration, gesture playback, and runtime logging. After calibration, preset gestures were replayed on the physical prototype to verify the complete workflow from PC-side commands to physical hand motion.

### 1.3 Report Organization

The rest of this report is organized as follows. Section 2 clarifies the boundary between the open-source DexHand resources and our own work. The following sections describe the mechanical implementation, electronics and firmware, PC-side interface, calibration and demonstration results, and preliminary simulation exploration. Finally, the report summarizes the current limitations and future work.

## 2 Open-Source Basis and Our Contributions

This project was built on the open-source DexHand platform. DexHand provided the reference mechanical structure, tendon-driven principle, BOM, and ROS2/URDF simulation resources. Our work focused on engineering reproduction and adaptation, including physical assembly, ESP32-S3 + PCA9685 control hardware, firmware, PC-side software, servo calibration, gesture playback, and simulation preparation.

Table 1 summarizes the boundary between the open-source basis and our independent work.

**Table 1:** Boundary between DexHand open-source resources and our independent work.

| Module                        | Open-source basis                                                                                            | Our independent work                                                                                                                                       |
|-------------------------------|--------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mechanical design             | Five-finger bionic layout, hand–wrist–forearm structure, tendon-driven transmission, and 3D-printable parts. | Completed physical assembly of fingers, palm, wrist, forearm, tendon routing, servo installation, and mechanical adjustment.                               |
| Materials and BOM             | Reference component list and assembly-related material information.                                          | Purchased, organized, and supplemented actual mechanical, electronic, wiring, and assembly materials according to real project needs.                      |
| Control hardware              | Original reference control path based on Arduino Nano RP2040 Connect and related electronics documentation.  | Built an <b>ESP32-S3 + PCA9685</b> control platform, completed 15-servo wiring, power distribution, and hardware integration.                              |
| Firmware                      | General servo-control idea and command-driven actuation logic.                                               | Developed ESP32-S3 firmware for Wi-Fi HTTP, USB serial debugging, command parsing, servo-table mapping, safe range limiting, and PWM output.               |
| PC-side software              | No desktop monitor was directly used in our final workflow.                                                  | Developed a <b>Tkinter-based monitor</b> for pose control, servo commands, action recording, range calibration, demonstration playback, and log display.   |
| Calibration and demonstration | Gesture control is supported as a general concept in the reference platform.                                 | Built a practical calibration workflow, obtained safe ranges for 15 servos, saved preset actions, and verified sequential playback and smooth transitions. |
| Simulation                    | ROS2/URDF resources and visualization basis.                                                                 | Reproduced ROS2 visualization and started URDF-to-MuJoCo XML conversion for future reinforcement-learning-based control exploration.                       |

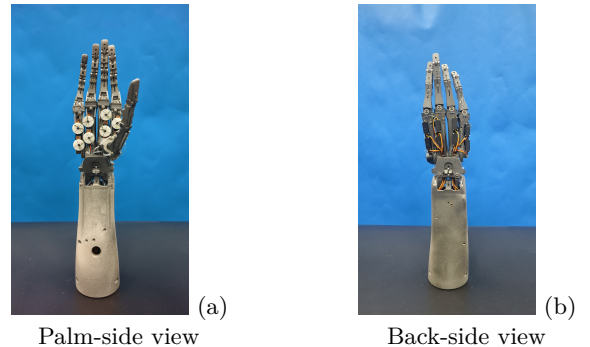
In summary, this report does not claim the original DexHand mechanical design as our own. The focus of our contribution is the reproduction, adaptation, integration, and validation of a DexHand-based system under our own hardware architecture.

### 3 Mechanical Implementation

#### 3.1 Overall Mechanical Structure

The mechanical implementation follows the DexHand hand–wrist–forearm layout. From distal to proximal, the mechanism is organized into finger modules, a palm/knuckle base, a wrist connector, and a forearm shell. This layout differs from a simple gripper: the hand uses multiple articulated digits, tendon transmission, elastic return elements, and distributed actuator placement to produce human-like grasping and gesture motion.

The five fingers provide the grasping interface. The palm fixes the finger bases, organizes the knuckle mechanisms, and guides the tendon paths. The wrist connects the hand to the forearm and provides orientation adjustment in the reference design. The forearm supplies the main structural volume for actuator installation, tendon routing, wiring, covers, and power/control hardware. During assembly, the open-source 3D-printed parts were converted into a real prototype by installing joints, springs, tendons, servo horns, tendon tubes, bearings, screws, and fasteners. The main mechanical challenge was to keep the long tendon paths smooth from actuator to joint while avoiding interference among adjacent tendons, wires, and printed shells.



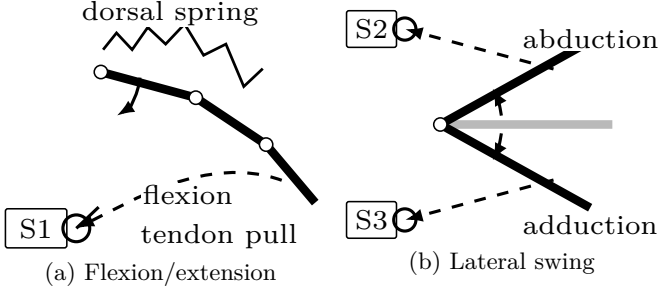
**Figure 1:** Overall mechanical structure of the assembled DexHand-based prototype. The hand consists of finger links, palm mechanism, wrist connector, forearm shell, tendons, servo spools, and internal wiring.

#### 3.2 Finger Structure

Each finger is built as a multi-link bionic mechanism. In the reference layout, a digit is divided into three printed link sections: the proximal phalanx (PLP), the middle phalanx (MLP), and the fingertip link. For the index, middle, ring, and little fingers, these links are connected to the palm through knuckle modules and correspond to the MCP, PIP, and DIP joint regions of a human finger. The different finger lengths in the reference design help approximate human-hand proportions. The thumb uses a different CMC base geometry so that it can oppose the other fingers, but it follows the same link-and-tendon principle.

The finger joints are designed to support flexion and extension, while the MCP region also supports lateral motion for abduction and adduction. During bending, a servo winds and pulls the palmar tendon so that the linked phalanges rotate toward flexion. A spring is connected along the dorsal side of the finger, providing the resting preload and rebound force needed for extension when the

tendon is released. For lateral motion, two servos pull opposed side tendons around the MCP/knuckle region, producing abduction and adduction. In the real prototype, finger motion quality depends strongly on joint alignment, tendon length, friction, and the installation angle of the servo horn.



**Figure 2:** Finger actuation principle. During flexion, a servo winds the palmar tendon and bends the linked phalanges; a dorsal spring supplies resting preload and rebound force for extension. During lateral motion, two servos pull opposed side tendons around the MCP/knuckle region to produce abduction and adduction.

### 3.3 Palm Structure

The palm is the central mechanical organizer of the hand. It fixes the knuckle modules, supports the finger-base motion, and provides passages for tendons and wires. In the DexHand concept, the palm area also contains spring and servo-related structures used for restoring force and lateral finger motion. Tendon guide tubes are used around the palm to protect tendons, reduce cable entanglement, and maintain stable routing when multiple fingers move at the same time.

Mechanically, the palm must balance compactness and maintainability. The available internal space is limited, but the palm still needs to accommodate tendon channels, fasteners, return elements, and finger-base motion. During assembly, we adjusted the tendon routing and checked that each finger could move independently without serious rubbing against neighboring parts.

### 3.4 Wrist Structure

The wrist connects the palm to the forearm and gives the hand an additional orientation adjustment capability. In the DexHand reference structure, the wrist assembly supports pitch and yaw motion, and the full reference mechanism also reserves a path for axial wrist rotation through a serial wrist actuator. In our report, the wrist is mainly described by pitch and yaw, corresponding to two rotational degrees of freedom relevant to the current mechanical discussion. The wrist also acts as an intermediate routing section: long tendons pass through the wrist from the forearm toward the palm and fingers.

Because the tendons cross the wrist region, the wrist structure must preserve tendon continuity while allowing hand rotation. This makes the wrist different from a simple rigid connector. If the tendon paths are too tight or poorly aligned, wrist motion can introduce additional friction or unwanted coupling into finger motion. Therefore, the wrist assembly requires careful alignment between the palm-side passages and the forearm-side actuator area.

### 3.5 Forearm Structure

The forearm provides the main support volume for actuator installation, tendon spools, tendon anchoring, wiring, and external covers. In the reference design, the forearm is also the natural place for the controller, wireless communication module, power conversion, and battery compartment. In the tendon-driven layout, moving a large part of the actuator and electronics volume into the forearm reduces the size and mass of the distal hand. This improves the compactness of the palm and fingers, but it also makes tendon routing longer and more sensitive to friction.

In the final prototype, the forearm assembly includes the printed forearm body, covers, servo mounting positions, tendon-spool related parts, and wiring space. The forearm-side servos pull long tendons to drive finger bending, while the palm-side mechanism is responsible for lateral finger motion in the DexHand concept. This distributed arrangement separates the compact hand structure from the actuator volume and makes the prototype closer to a human-like hand-arm layout.

### 3.6 Degrees of Freedom and Tendon-Driven Principle

The DexHand mechanical design uses a high-degree-of-freedom finger layout. As summarized in the final presentation, each finger has DIP flexion/extension, PIP flexion/extension, MCP flexion/extension, and MCP abduction/adduction. This gives four mechanical degrees of freedom per finger. For five fingers, the hand therefore has about 20 finger DOF. The wrist adds pitch and yaw, giving another two DOF in the reference mechanical structure.

**Table 2:** Mechanical DOF summary of the DexHand reference structure.

| Part         | DOF | Motion description                                                                            |
|--------------|-----|-----------------------------------------------------------------------------------------------|
| Each finger  | 4   | DIP flexion/extension, PIP flexion/extension, MCP flexion/extension, MCP abduction/adduction. |
| Five fingers | 20  | Five-finger dexterous grasping and gesture motion.                                            |
| Wrist        | 2   | Pitch and yaw orientation adjustment.                                                         |

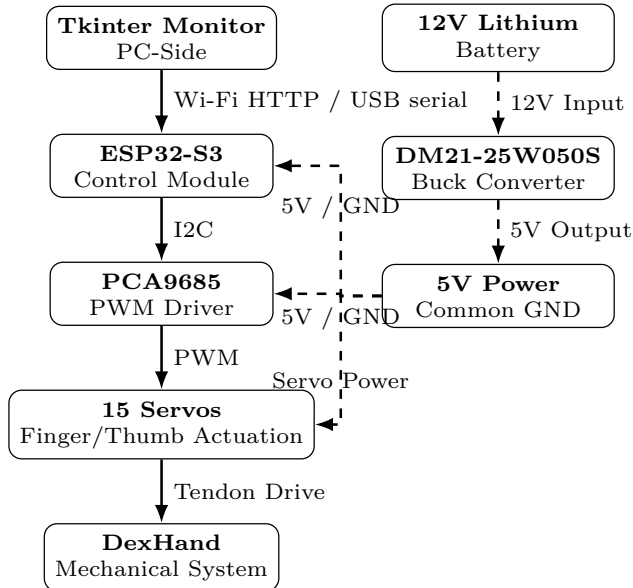
The actuation principle is tendon-driven. A servo rotates a horn or spool to wind or release a tendon. When the tendon is pulled, its effective length becomes shorter and the corresponding joint bends or swings. When the tendon is released, the spring provides restoring force and rebound motion, helping the joint return toward its initial position. A long tendon can pass through the forearm, wrist, palm, and finger, so the actuator does not have to be placed directly at the joint.

This mechanism keeps the distal side of the hand compact and lightweight, but it also introduces practical issues such as tendon friction, routing sensitivity, backlash, and inconsistent return behavior. During assembly, these issues were handled by checking joint alignment, reorganizing tendons and wires, reducing sharp bends in the tendon path, and improving servo and mechanical part installation. By the final stage, the main mechanical assembly was complete and ready for calibration and motion validation through the electronics and software system.

## 4 Electronics and Firmware

### 4.1 Overall Control and Power Architecture

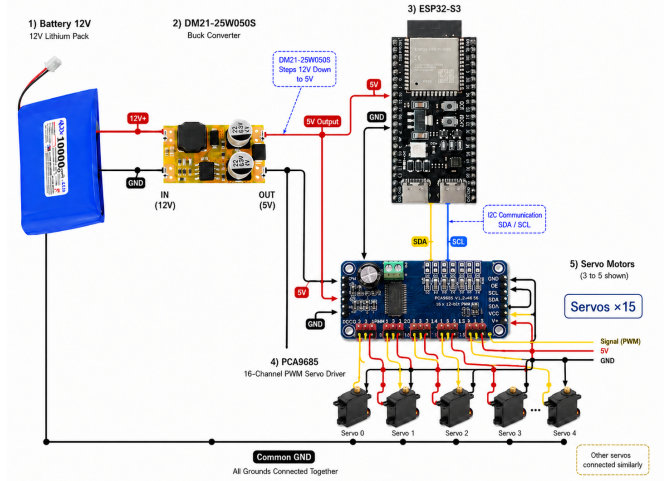
The electronics system was designed to convert commands from the PC-side interface into controllable servo motions on the physical DexHand prototype. Compared with the original DexHand reference electronics, our implementation uses an **ESP32-S3 control module** as the main embedded controller and a **PCA9685 PWM driver module** for multi-channel servo signal generation. The overall architecture separates high-level command input, embedded processing, PWM generation, actuator driving, and power distribution.



**Figure 3:** Overall control and power architecture of the ESP32-S3 and PCA9685-based DexHand system.

As shown in Fig. 3, the **Tkinter-based PC-side monitor** sends control commands to the ESP32-S3 through **Wi-Fi HTTP** or **USB serial**. The ESP32-S3 processes each command and communicates with the PCA9685 through the **I2C bus**. The PCA9685 then generates PWM signals for the servo motors, which actuate the tendon-driven finger mechanism.

The power path is supplied by a **12 V lithium battery**. A **DM21-25W050S DC-DC buck converter** steps the battery voltage down to **5 V**, which is distributed to the ESP32-S3, PCA9685 control board, and servo power rail. All electronic modules share a **common ground** to provide a consistent signal reference. A physical wiring diagram of the implemented electronics system is shown in Fig. 4.



**Figure 4:** Physical wiring diagram of the implemented electronics system.

### 4.2 Electrical Connection and Servo Channel Mapping

The electrical connection was organized according to two requirements: reliable power distribution and stable servo signal transmission. The ESP32-S3 is connected to the PCA9685 through SDA and SCL lines for I2C communication. The PCA9685 module uses **VCC** for logic supply and **V+** for the external servo power input. Each servo is connected to one PCA9685 output channel through its signal line, while its power and ground are connected to the 5 V power rail and common ground.

A fixed channel mapping was used to keep the firmware, PC-side monitor, and calibration process consistent. In the current prototype, CH0-CH14 are assigned to the main finger and thumb actuation servos, as shown in Table 3.

**Table 3:** Detailed PCA9685 channel mapping for the implemented 15-servo DexHand prototype.

| Channel | Connected Servo | Function Description          |
|---------|-----------------|-------------------------------|
| CH0     | Index lower     | Index lower tendon actuation  |
| CH1     | Index upper     | Index upper tendon actuation  |
| CH2     | Index tip       | Index distal/tip actuation    |
| CH3     | Middle lower    | Middle lower tendon actuation |
| CH4     | Middle upper    | Middle upper tendon actuation |
| CH5     | Middle tip      | Middle distal/tip actuation   |
| CH6     | Ring lower      | Ring lower tendon actuation   |
| CH7     | Ring upper      | Ring upper tendon actuation   |
| CH8     | Ring tip        | Ring distal/tip actuation     |
| CH9     | Little lower    | Little lower tendon actuation |
| CH10    | Little upper    | Little upper tendon actuation |
| CH11    | Little tip      | Little distal/tip actuation   |
| CH12    | Thumb left      | Thumb lateral actuation       |
| CH13    | Thumb right     | Thumb lateral actuation       |
| CH14    | Thumb flexion   | Thumb flexion/tip actuation   |

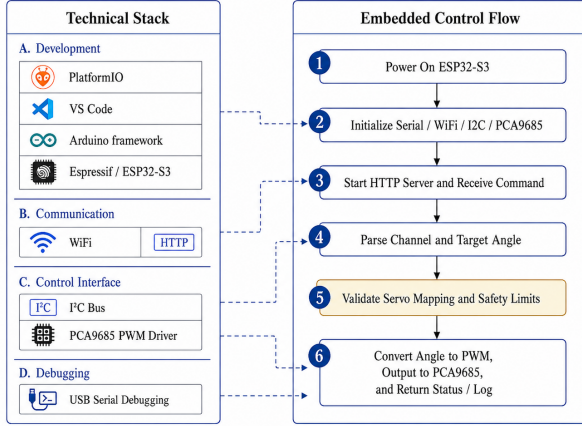
The original DexHand firmware defines additional servo indices for thumb rotation and wrist motion, while the current implementation focuses on the 15 channels used for the main finger and thumb motions. This channel assignment matches the current hardware prototype and simplifies subsequent calibration and demonstration playback.

### 4.3 Firmware Workflow

The firmware was developed for the **ESP32-S3 control module** using **PlatformIO** in **VS Code** with the **Arduino framework**. It integrates Wi-Fi communication,

USB serial debugging, I2C communication, and PCA9685-based PWM output. Its main role is to translate PC-side commands into low-level servo control signals.

After power-on, the ESP32-S3 initializes serial communication, Wi-Fi, I2C, and the PCA9685 driver. It then starts an HTTP server and loads the predefined servo mapping table and safe motion ranges for each channel. These definitions allow each received command to be associated with a specific physical actuator.



**Figure 5:** Firmware workflow of the ESP32-S3-based DexHand control system.

During operation, each command is parsed into a target channel and a target angle. The firmware checks whether the channel is valid, constrains the target angle within the calibrated safe range, converts the result into a PWM signal, and sends it to the corresponding PCA9685 channel. Execution status is returned through serial logs or HTTP responses for debugging and verification.

#### 4.4 Communication Interface

The communication interface connects the Tkinter-based monitor with the **ESP32-S3 control module**. The system supports two command channels: **Wi-Fi HTTP communication** for normal PC-side control and **USB serial communication** for firmware uploading, debugging, and direct command testing.

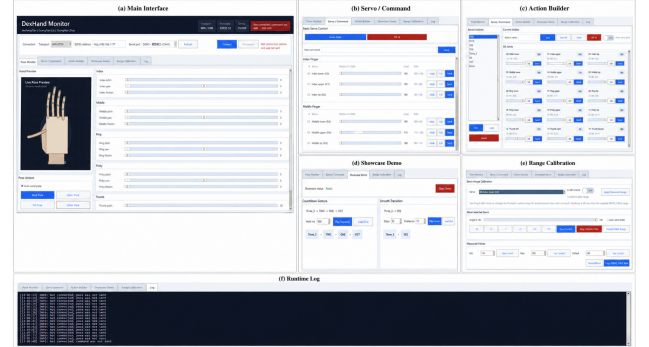
Both interfaces follow the same command-based control logic. After receiving a command, the ESP32-S3 extracts the target channel and target value, then passes them to the firmware workflow described above. More advanced functions, such as pose adjustment, range calibration, action recording, and demonstration playback, are implemented in the PC-side software and are discussed in later sections.

## 5 PC-Side Interface

### 5.1 Design Purpose and Interface Overview

To support hardware bring-up, servo tuning, pose testing, action recording, calibration, and demonstration playback, a **Tkinter-based PC-side monitor** was developed for the ESP32-S3-based DexHand prototype. The interface connects user operations with the embedded firmware and the physical servo system, replacing manual low-level command input with a more transparent and repeatable graphical workflow.

The monitor is organized as a multi-tab desktop interface. Each tab corresponds to a specific stage of the development and testing process, from single-servo verification to full-hand gesture demonstration. In this way, the PC-side system acts not only as a control panel but also as an integrated debugging and presentation tool for the complete prototype.



**Figure 6:** Overview of the Tkinter-based PC-side monitor and its main functional tabs.

**Table 4:** Main functional modules of the Tkinter monitor.

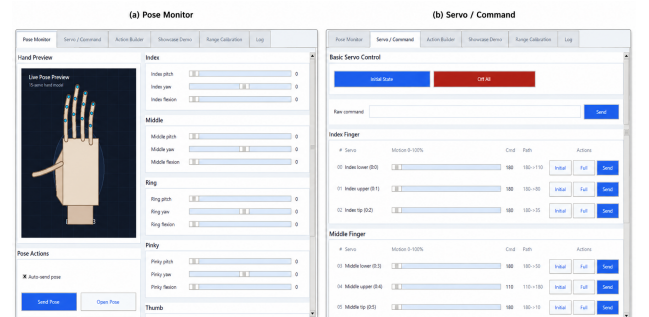
| Module            | Main function                                                    |
|-------------------|------------------------------------------------------------------|
| Pose Monitor      | Real-time hand pose adjustment and live preview                  |
| Servo / Command   | Single-servo control, command sending, and actuator verification |
| Action Builder    | Full 15-servo action editing, saving, and replay                 |
| Showcase Demo     | Gesture sequence playback and smooth-transition demonstration    |
| Range Calibration | Servo limit measurement and default-position calibration         |
| Log / Raw Command | Runtime feedback and low-level firmware debugging                |

### 5.2 Pose and Servo Control

The PC-side monitor provides two complementary control layers: **servo-level control** and **pose-level control**. Servo-level control directly operates the calibrated servo channels, while pose-level control describes hand motion through higher-level posture variables. Table 5 summarizes their relationship, and Fig. 7 shows the corresponding GUI panels.

**Table 5:** Control layers used in the PC-side interface.

| Layer               | Input                                           | Output and Usage                                                                                                                  |
|---------------------|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Servo-level control | Servo command angle or 0–100% travel            | Converted to calibrated servo commands and PCA9685 PWM outputs; used for actuator testing, calibration, and saved action playback |
| Pose-level control  | 15-DOF pose vector with pitch, yaw, and flexion | Mapped to lower, upper, and tip servo groups; used for intuitive whole-hand posture adjustment                                    |



**Figure 7:** Pose-level and servo-level control interfaces. The pose interface controls the hand through pitch, yaw, and flexion variables, while the servo interface directly adjusts the calibrated 15-channel actuator commands.

For servo-level control, the value shown in the interface is treated as a **logical command angle**, rather than the absolute mechanical angle of the servo output shaft. The requested angle is first constrained by the calibrated range of the corresponding servo:

$$\theta_c = \text{clip}(\theta_r, \theta_{\min}, \theta_{\max}), \quad (1)$$

where  $\theta_r$  is the requested angle,  $\theta_c$  is the constrained command angle, and  $\theta_{\min}$  and  $\theta_{\max}$  are the calibrated limits stored in the servo table. If the servo is configured as inverted, the command is mirrored before PWM generation:

$$\theta_p = \begin{cases} 180^\circ - \theta_c, & \text{if inverted,} \\ \theta_c, & \text{otherwise.} \end{cases} \quad (2)$$

The processed command  $\theta_p$  is then converted into the PCA9685 output value. In the current implementation, the servo pulse width is mapped from 0–180° to 700–2300  $\mu\text{s}$  and then converted into a 12-bit PWM tick value:

$$\begin{aligned} u &= 700 + \frac{\theta_p}{180}(2300 - 700), \\ T_{\text{PWM}} &= \frac{10^6}{50}, \\ N &= \frac{u}{T_{\text{PWM}}} \times 4096. \end{aligned} \quad (3)$$

Here,  $u$  is the pulse width in microseconds,  $T_{\text{PWM}}$  is the PWM period at 50 Hz, and  $N$  is the PCA9685 tick value. This mapping keeps the PC-side command scale consistent even when different servos have different installation directions and calibrated ranges.

The Servo/Command page also provides a normalized 0–100% travel representation. Instead of directly exposing only raw angle values, the slider describes the relative motion from the stored default position to the calibrated motion endpoint:

$$\theta_{\text{cmd}} = \theta_0 + (\theta_e - \theta_0) \frac{p}{100}, \quad (4)$$

where  $\theta_0$  is the default command angle,  $\theta_e$  is the calibrated motion endpoint, and  $p$  is the slider percentage. Thus,  $p = 0$  corresponds to the initial state and  $p = 100$  corresponds to the full calibrated travel.

For pose-level control, the hand posture is represented by a 15-DOF vector:

$$\mathbf{q} = [q_0, q_1, \dots, q_{14}]^T, \quad (5)$$

where the DOFs correspond to the pitch, yaw, and flexion values of the index, middle, ring, little finger, and thumb. For each non-thumb finger, the pose variables and servo commands can be written as:

$$\mathbf{q}_f = [\theta_{\text{pitch}}, \theta_{\text{yaw}}, \theta_{\text{flex}}]^T, \quad \mathbf{s}_f = [s_{\text{lower}}, s_{\text{upper}}, s_{\text{tip}}]^T. \quad (6)$$

In the implemented mapping, pitch mainly affects the

lower and upper servos, yaw is generated by differential motion between these two servos, and flexion mainly drives the tip servo. A simplified form of this mapping is:

$$\begin{aligned} s_{\text{lower}} &= F_l(\theta_{\text{pitch}}) + \Delta_{\text{yaw}} + \Delta_{\text{flex}}, \\ s_{\text{upper}} &= F_u(\theta_{\text{pitch}}) - \Delta_{\text{yaw}} + \Delta_{\text{flex}}, \\ s_{\text{tip}} &= F_t(\theta_{\text{flex}}), \end{aligned} \quad (7)$$

where  $F_l(\cdot)$ ,  $F_u(\cdot)$ , and  $F_t(\cdot)$  are range-mapping functions from pose variables to servo command ranges. The yaw contribution is reduced when the finger is highly flexed:

$$\Delta_{\text{yaw}} = \bar{\theta}_{\text{yaw}}(1 - \bar{\theta}_{\text{flex}})k_{\text{yaw}}, \quad (8)$$

where  $\bar{\theta}_{\text{yaw}}$  and  $\bar{\theta}_{\text{flex}}$  are normalized yaw and flexion values, and  $k_{\text{yaw}}$  is the yaw gain. This reduces lateral motion when the finger is already bent and helps avoid mechanical conflict in the tendon-driven structure.

The thumb follows the same high-level idea, but its pitch, yaw, and flexion variables are mapped to the thumb left, thumb right, and thumb flexion servos. To better fit the current thumb mechanism, the yaw command uses a threshold-based folded mapping:

$$\theta'_{\text{yaw}} = \begin{cases} \theta_{\text{yaw}}, & \theta_{\text{yaw}} < 30^\circ, \\ 30^\circ - 2(\theta_{\text{yaw}} - 30^\circ), & \theta_{\text{yaw}} \geq 30^\circ. \end{cases} \quad (9)$$

The processed yaw value is then mapped to the thumb-right servo command range.

Overall, servo-level control provides predictable actuator commands for testing, calibration, and action playback, while pose-level control provides a more intuitive way to describe coordinated hand posture. The two layers together support both low-level debugging and higher-level hand motion control.

### 5.3 Action Builder and Demonstration Playback

To support reusable gesture generation and presentation-oriented playback, the PC-side monitor includes an **Action Builder** module and a **Showcase Demo** module. These modules turn manually adjusted joint states into repeatable full-hand actions. In the current implementation, one action is represented as a complete 15-servo command frame:

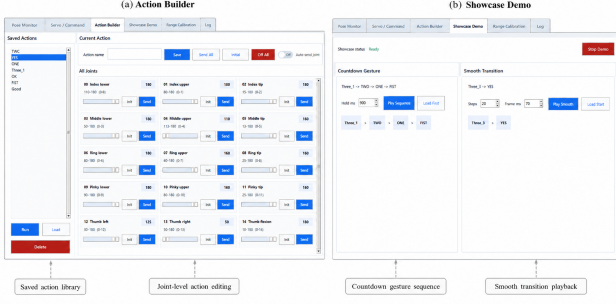
$$\mathbf{a} = [\theta_0, \theta_1, \dots, \theta_{14}]^T, \quad (10)$$

where  $\theta_i$  denotes the command angle of the  $i$ -th servo channel. This frame-based representation preserves the full actuator state of the hand and is suitable for gesture storage, replay, and presentation.

On the **Action Builder** page, the user directly edits the 15 servo command values and saves them as a named action preset. During playback, the stored action is sent to the firmware as a compact **servoframe** command:

$$\text{servoframe} = [\theta_0, \theta_1, \dots, \theta_{14}]. \quad (11)$$

Compared with sending individual servo commands one by one, this frame-based command groups all actuator targets into one synchronized action. Figure 8 shows the main interfaces of the Action Builder and Showcase Demo modules.



**Figure 8:** Main interfaces of the Action Builder and Showcase Demo modules. The Action Builder is used to edit and save full 15-servo action frames, while the Showcase Demo is used to replay saved actions for presentation.

The **Showcase Demo** module reuses saved action presets in two ways. The first is **sequential playback**, where several predefined actions are executed in order:

$$\begin{aligned} \text{Three} &\rightarrow \text{Two} \rightarrow \text{One} \\ &\rightarrow \text{Fist.} \end{aligned} \quad (12)$$

In this mode, the monitor loads the corresponding saved servo frames and sends them to the firmware one by one with a selected playback interval.

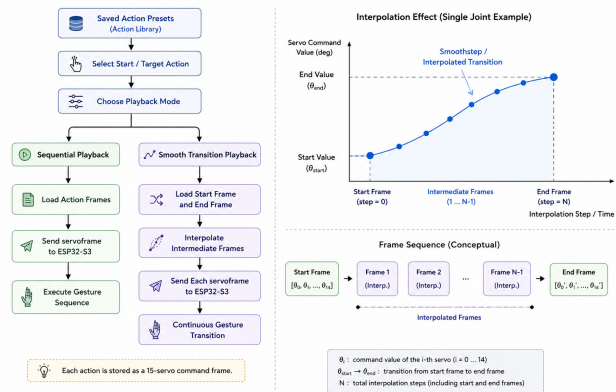
The second mode is **smooth-transition playback**, which generates intermediate frames between a start action and a target action. Let  $\mathbf{a}_s$  and  $\mathbf{a}_e$  denote the start and end servo frames. For the  $k$ -th interpolation step,

$$t_k = \frac{k}{K}, \quad \alpha_k = t_k^2(3 - 2t_k), \quad (13)$$

where  $K$  is the total number of interpolation steps and  $\alpha_k$  is a smoothstep weight. The intermediate servo frame is then computed as:

$$\mathbf{a}_k = \mathbf{a}_s + \alpha_k(\mathbf{a}_e - \mathbf{a}_s). \quad (14)$$

Each interpolated frame is sent as a new **servoframe**, allowing the hand to move continuously from one gesture to another. Figure 9 illustrates the complete demonstration playback workflow and the interpolation effect.



**Figure 9:** Workflow of demonstration playback. Saved actions are replayed either as discrete gesture sequences or as smooth transitions generated from interpolated servo frames.

Through these two modules, the PC-side monitor extends the system from manual actuator control to repeatable gesture generation and presentation-oriented play-

back. This design improves both the reproducibility and the presentation quality of the DexHand prototype.

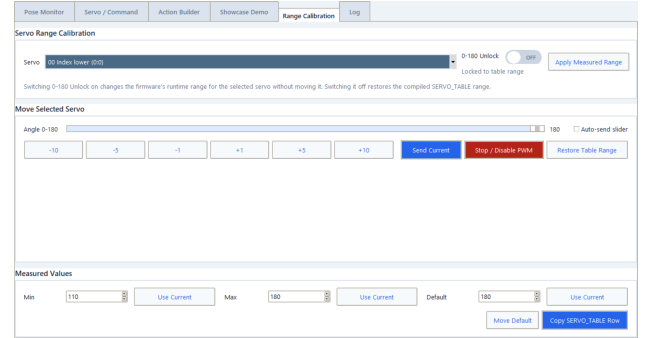
## 5.4 Range Calibration and Debugging Support

Because the DexHand prototype is driven by tendons and manually assembled mechanical parts, the safe motion range of each servo is affected by tendon routing, servo horn installation, joint friction, and mechanical interference. Therefore, a dedicated **Range Calibration** tab was implemented in the PC-side monitor to determine a reliable command range for each servo before full-hand control and demonstration playback.

For each servo channel, the calibration process records three key values:

$$\mathcal{R}_i = [\theta_{i,\min}, \theta_{i,\max}, \theta_{i,0}], \quad (15)$$

where  $\theta_{i,\min}$  and  $\theta_{i,\max}$  are the safe minimum and maximum command angles of the  $i$ -th servo, and  $\theta_{i,0}$  is the default angle corresponding to the initial mechanical posture. During calibration, the interface can temporarily unlock the runtime range of a selected servo to 0–180° so that its physical limit can be tested with small step movements. After the safe limits are measured, the values can be copied back into the firmware servo table, allowing later commands to be automatically constrained within the calibrated range.



**Figure 10:** Range calibration and debugging interface. The calibration panel is used to measure servo limits and default values, while the command and log panels support low-level firmware verification.

In addition to range calibration, the monitor provides raw command input, runtime log display, and an **Off All** function that disables PWM outputs for all servo channels when the mechanism needs to be released during testing. Together, these functions form a practical tuning loop:

$$\begin{aligned} \text{test motion} &\rightarrow \text{measure safe range} \rightarrow \\ &\text{update servo table} \rightarrow \text{verify motion.} \end{aligned} \quad (16)$$

This loop improves the reliability of later pose control, saved-action playback, and demonstration sequences.

## 6 Calibration and Demonstration

### 6.1 Servo Calibration Results

Based on the range calibration procedure described in the PC-side interface section, the final safe command ranges of all 15 servos were obtained and written into the firmware servo table. These calibrated values define the allowable command interval, default posture command, and inversion setting of each actuator. They serve as the

basis for subsequent pose control, saved-action playback, and demonstration sequences.

All servos in the current prototype are connected to the same PCA9685 driver board with address 0x40. The calibrated results are summarized in Table 6. The **Range** column represents the safe command-angle interval used by the firmware, the **Default** column corresponds to the initial posture of the hand, and the **Inverted** column compensates for servo installation direction during PWM generation.

The calibration results show that the allowable command ranges are not identical across servos, which reflects differences in tendon routing, servo horn installation, joint friction, and mechanical limits after assembly. After the calibration table was finalized, all saved gestures and demonstration actions were generated within these command limits, improving playback repeatability and reducing the risk of tendon over-tension or mechanical interference during continuous gesture execution.

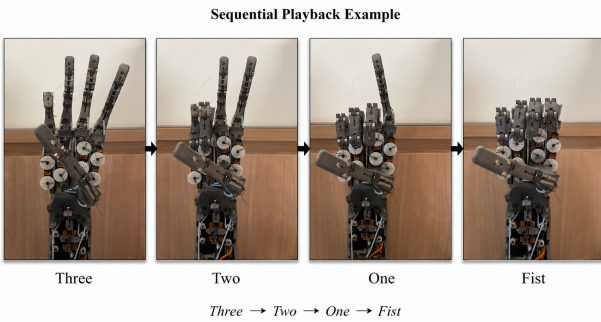
### 6.2 Demonstration Playback

After the servo calibration table was finalized, several preset gestures were created and saved using the PC-side interface. Each gesture corresponds to a complete 15-servo command frame, which can be replayed by sending the saved frame to the ESP32-S3 firmware. The demonstration mainly includes sequential gesture playback and smooth-transition playback.

In sequential playback, saved gestures are executed in a fixed order, such as

$$\begin{aligned} \text{Three} &\rightarrow \text{Two} \rightarrow \text{One} \\ &\rightarrow \text{Fist}. \end{aligned} \quad (17)$$

In smooth-transition playback, intermediate frames are generated between two saved gestures, such as the transition from *Three* to *YES*, so that the hand motion changes more continuously.



**Figure 11:** Continuous frames of the preset gesture demonstration after servo range calibration.

As shown in Fig. 11, the calibrated prototype can replay saved gestures and perform continuous transitions between different hand poses. This demonstrates the usability of the saved-action playback workflow for final presentation and repeated testing.

### 6.3 Results and Discussion

The calibration and demonstration results verify the basic end-to-end workflow of the implemented DexHand prototype. The PC-side monitor can generate and replay saved action commands, the ESP32-S3 firmware can

parse these commands and output PWM signals through the PCA9685 driver, and the calibrated servos can drive the tendon-based mechanical structure to perform visible hand gestures. Based on the calibrated servo table, the preset actions could be replayed repeatedly, while smooth-transition playback reduced abrupt changes between different gestures.

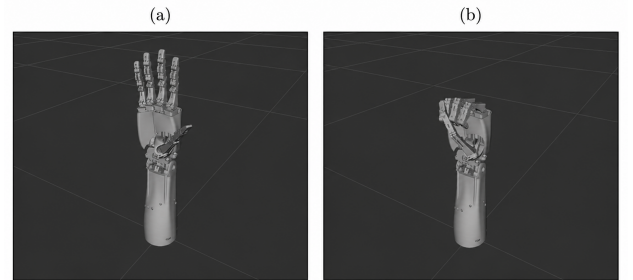
However, the current system is still mainly based on open-loop servo control. The final gesture quality is affected by tendon tension, servo horn installation, joint friction, and mechanical assembly errors. Since the prototype does not yet include direct joint-angle or force feedback, the controller cannot automatically correct motion deviations during execution. Future work may improve the system by adding sensing feedback, refining the mechanical adjustment, expanding the gesture library, and introducing higher-level intelligent control methods.

## 7 Simulation and Intelligent Control Exploration

This part of the project was carried out as a preliminary exploration for future intelligent control. Compared with the hardware assembly, firmware development, and PC-side implementation, the simulation and intelligent control work is still at an early stage. Its main purpose is to understand the DexHand simulation workflow, verify the basic behavior of the model, and prepare the environment for later reinforcement-learning-based control experiments.

### 7.1 ROS2 Simulation Validation

The first step was to reproduce the ROS2-based simulation workflow of DexHand. The hand model was launched and visualized in RViz, and basic open-hand and closed-hand poses were tested to verify that the robot description, launch files, and joint control pipeline worked correctly. This process helped us understand the relationship between the URDF model, ROS2 packages, visualization interface, and joint motion commands.



**Figure 12:** ROS2 simulation and visualization of the Dex-Hand model.

The ROS2 validation does not directly control the physical prototype, but it provides a useful reference for understanding the kinematic structure and joint behavior of the hand. It also reduces uncertainty before further simulation-based control development.

### 7.2 MuJoCo and Intelligent Control Preparation

In addition to ROS2 visualization, we started preparing a MuJoCo-based simulation environment for future control

**Table 6:** Calibrated servo ranges of the implemented DexHand prototype.

| Index | Servo Name    | Driver | Channel | Range   | Default | Inverted |
|-------|---------------|--------|---------|---------|---------|----------|
| 0     | Index lower   | 0      | 0       | 110–180 | 180     | True     |
| 1     | Index upper   | 0      | 1       | 80–180  | 180     | False    |
| 2     | Index tip     | 0      | 2       | 35–180  | 180     | True     |
| 3     | Middle lower  | 0      | 3       | 50–180  | 180     | True     |
| 4     | Middle upper  | 0      | 4       | 110–180 | 110     | False    |
| 5     | Middle tip    | 0      | 5       | 10–180  | 180     | True     |
| 6     | Ring lower    | 0      | 6       | 80–180  | 180     | False    |
| 7     | Ring upper    | 0      | 7       | 40–180  | 180     | True     |
| 8     | Ring tip      | 0      | 8       | 25–180  | 180     | True     |
| 9     | Pinky lower   | 0      | 9       | 90–180  | 180     | False    |
| 10    | Pinky upper   | 0      | 10      | 30–180  | 180     | True     |
| 11    | Pinky tip     | 0      | 11      | 25–180  | 180     | True     |
| 12    | Thumb left    | 0      | 12      | 30–180  | 125     | True     |
| 13    | Thumb right   | 0      | 13      | 50–180  | 50      | False    |
| 14    | Thumb flexion | 0      | 14      | 10–180  | 180     | True     |

experiments. The current workflow is to export the DexHand model from the ROS URDF format and convert it into a MuJoCo-compatible XML model. This conversion is necessary because MuJoCo provides a physics simulation environment better suited to later reinforcement learning and dexterous manipulation tasks.

#### ROS URDF Model → Model Conversion → MuJoCo XML → Control Environment

**Figure 13:** Model conversion pipeline for future MuJoCo-based control experiments.

At the same time, we studied the basic formulation of reinforcement learning for dexterous hand control, including the definitions of state, action, and reward. At this stage of the project, the RL-related work was mainly used to build background knowledge and prepare the simulation environment. Future work can further define task-specific observations, action spaces, reward functions, and training pipelines based on the calibrated physical prototype and the MuJoCo model.

## 8 Limitations and Future Work

Although the prototype verifies the main workflow from PC-side commands to physical gesture playback, several limitations remain. The current system is still mainly based on open-loop servo control, and the final hand posture can be affected by tendon friction, servo installation error, joint friction, and assembly tolerance. In addition, the demonstrated gestures are based on manually calibrated and predefined action frames, so the system does not yet support adaptive motion generation or automatic correction during execution.

Future work will focus on improving mechanical reliability, control feedback, and intelligent interaction. Mechanically, tendon routing and joint alignment should be

further refined. For control, joint-angle, force, current, or tactile feedback can be added to support closed-loop motion. On the software side, the action library, calibration workflow, and logging system can be improved. In addition, the MuJoCo simulation environment should be completed to support future reinforcement learning, voice control, or natural-language gesture interaction.

**Table 7:** Main limitations and future work.

| Aspect              | Future Direction                                                |
|---------------------|-----------------------------------------------------------------|
| Mechanical system   | Improve tendon routing, joint alignment, and assembly accuracy. |
| Control feedback    | Add sensing feedback for closed-loop motion control.            |
| Software tools      | Refine calibration, logging, and action management.             |
| Intelligent control | Complete MuJoCo setup and explore RL or language-based control. |

## Acknowledgements

The team acknowledges the open-source DexHand project and its community resources, which provided the reference mechanical design and simulation basis for this course project.

## References

- [1] DexHand Official Website. *DexHand: Open-source dexterous hand platform*. Available: <https://www.dexhand.org/>
- [2] Open Robotics. *ROS 2 Documentation*. Available: <https://docs.ros.org/>
- [3] Google DeepMind. *MuJoCo Documentation*. Available: <https://mujoco.readthedocs.io/>
- [4] Espressif Systems. *ESP32-S3 Series Datasheet*.

## Appendix A: Bill of Materials

The bill of materials in Table A.1 is summarized from the final presentation cost sheet. It records the purchased mechanical, actuator, electronic, wiring, and auxiliary materials used during the project. Some purchased items reflect the reference BOM and early hardware route; the final implemented control architecture was later adapted to ESP32-S3 + PCA9685 as described in the main text.

**Table A.1:** Project BOM and cost summary based on the final presentation.

| Category     | Component                                      | Cost           | Ratio       |
|--------------|------------------------------------------------|----------------|-------------|
| Finger       | Thumb                                          | 36.41          | 1.53%       |
|              | Index                                          | 50.68          | 2.12%       |
|              | Middle                                         | 81.34          | 3.41%       |
|              | Ring                                           | 48.41          | 2.03%       |
|              | Little                                         | 50.77          | 2.13%       |
| Wrist        | Wrist assembly                                 | 102.67         | 4.30%       |
| Forearm      | Forearm printed parts                          | 45.05          | 1.89%       |
| Forearm      | Forearm shell                                  | 313.00         | 13.12%      |
| Transmission | Tendon spools                                  | 34.00          | 1.43%       |
| Controller   | Arduino Nano RP2040 Connect (1 pc)             | 200.00         | 8.38%       |
| Actuator     | ES3301 servo ( $16 \times 26.1$ )              | 417.60         | 17.51%      |
|              | FEETECH SCS2332 ( $2 \times 148$ )             | 296.00         | 12.41%      |
|              | FEETECH SCS15 (1 pc)                           | 83.00          | 3.48%       |
| Tendon       | Fishing line No. 8 ( $1 \times 88.4$ )         | 88.40          | 3.71%       |
| Hardware     | Other reference-BOM materials                  | 142.38         | 5.97%       |
| Hardware     | Additional materials outside the reference BOM | 395.55         | 16.58%      |
| <b>Total</b> |                                                | <b>2385.26</b> | <b>100%</b> |

*Note:* The “Other reference-BOM materials” line includes items such as a 4 cm  $\times$  6 cm Arduino prototype board, 2.54 mm female headers, 2.54 mm pin headers, cylindrical pin sets, M2 screws and washers, M2 threaded inserts, wire, bearings, and M3 fasteners. The “Additional materials” line records extra materials purchased during real assembly and debugging beyond the reference BOM.